



Towards Automated Tool Creation

The Problem: Manual Tool Creation

Tools are central to modern agentic systems, yet their creation remains largely manual, creating a critical bottleneck.



Ad-hoc Implementations

Lack of consistent quality or structure across manual hand-coded tools.



Poor Scalability

Developers cannot keep pace with the thousands of APIs required for modern systems.



Limited Autonomy

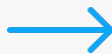
Agents remain restricted to a small set of pre-built, human-defined tools.

Paradigm Shift: From Tool Use to Tool Synthesis

Prior Focus

Tool Use

Can agents call the right functions? AI benchmarking has focused heavily on this execution layer.



Emerging Capability

Tool Synthesis

Can agents build the tools themselves?
A more fundamental capability for autonomous system scaling.

Related Work



Tool Use Benchmarks

ToolBench, API-Bank, MCP-Universe.
Evaluate correct tool calling,
assuming tools already exist.



Code Generation

SWE-bench, HumanEval. Focus on
single-function or PR-level tasks, no
live API execution.



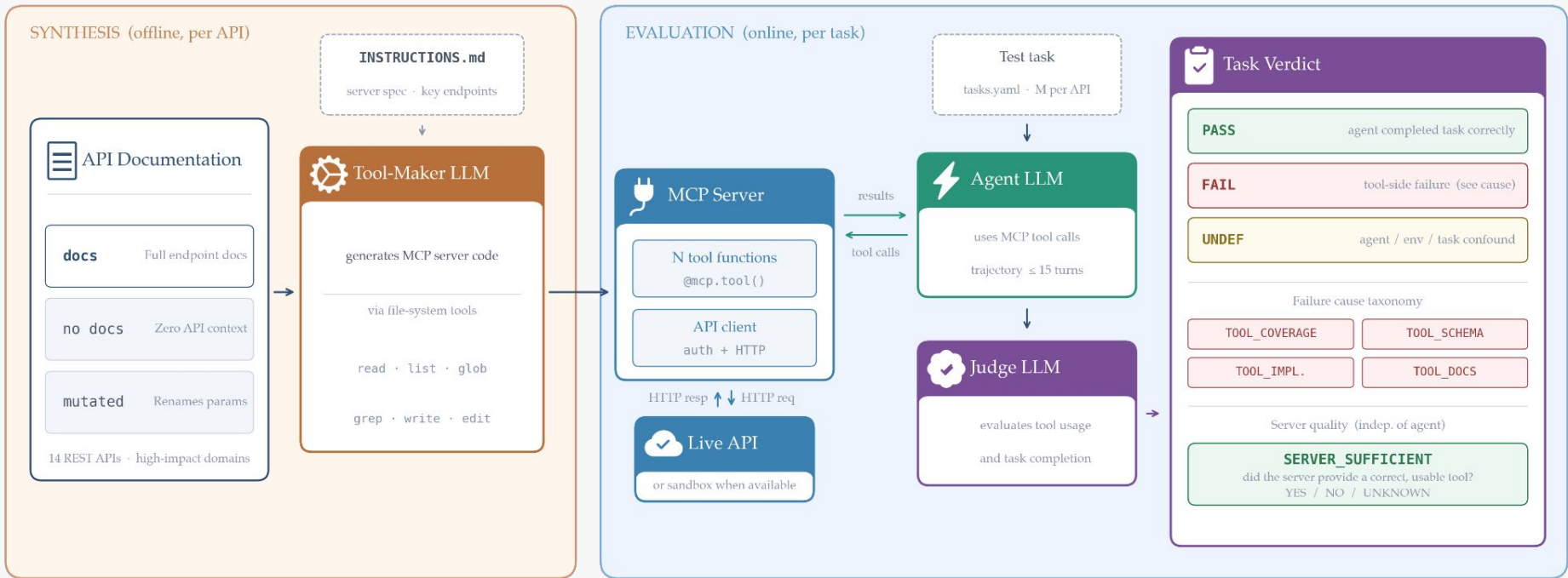
Agent Reasoning

MCPAgentBench, GAIA2, WebArena.
Evaluate high-level behaviour, not
tool construction.

Research Gap:

No prior work evaluates agentic tool-making against realistic tasks.

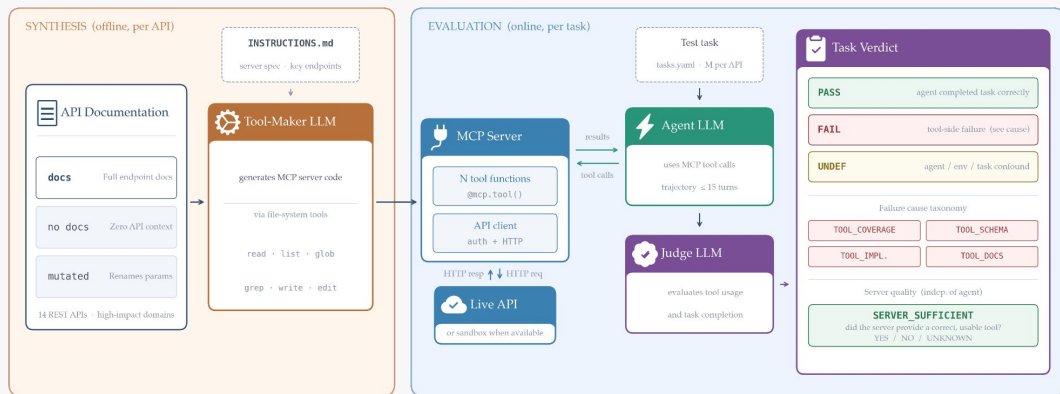
Our Approach: Overview



Synthesis Loop

An agentic loop with:

- **Basic toolset:** read, list, write, glob, grep, edit;
- Model reads **INSTRUCTIONS.md**, explores **docs** (if available), and **writes deliverables** iteratively.
- **Typical run** (GPT 5.2): 15–25 turns, 1–10 min, tokens 14K–2.4M in / 3K–27K out
no docs: 14K–24K in, ~1 min;
docs: 200K–2.4M in, 2–10 min.



Toolset Synthesis Example

Input: Task Specification

Task:

Build an MCP Server for eBay Buy API

What You Have:

- 28 API docs in docs/
- Env: APP_ID, CERT_ID

Deliverables:

- server.py (entry point)
- grounding.json (map tool to doc)
- requirements.txt

Synthesis Agent Trace

```
[turn 1] read_file(TASK.md)
[turn 1] glob(docs/*.md)
[turn 2] read_file(docs/browse.md)
[turn 4] write_file(browse.py)
[turn 7] write_file(order.py)
[turn 9] write_file(server.py)
[turn 11] write_file(grounding.json)
...
```

813K input / 11K output tokens
Time: 190s (GPT-5.2)

Generated Output

server.py (Python):

```
from mcp.server.fastmcp import
FastMCP
from generated_tools.client import
EbayClient
from generated_tools import
browse, deal, feed, order

mcp = FastMCP("ebay-buy-api")
client = EbayClient()
```

```
@mcp.tool()
def browse_search(
    q: Optional[str] = None,
```

requirements.txt:

```
fastmcp==3.2.4
requests==2.32.3
```

grounding.json:

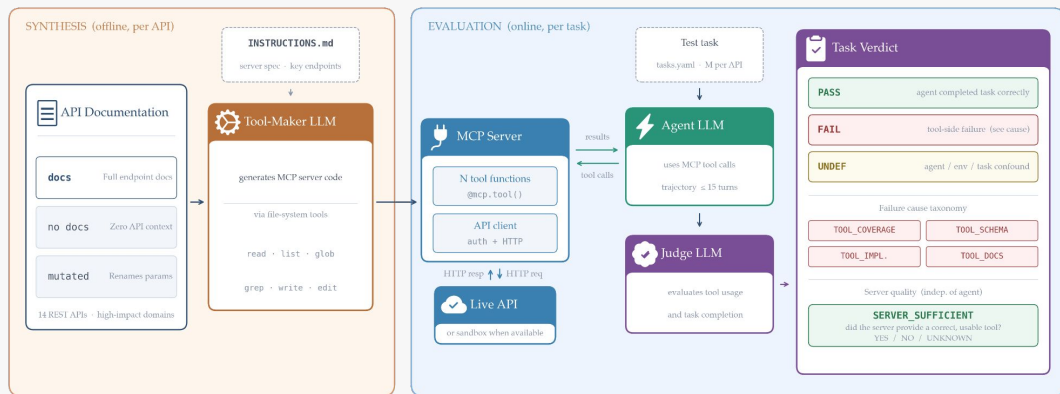
```
{
  "browse_search": {
    "endpoint": "GET /search"
  }
}
```

Evaluation Framework

For each task, an agentic loop:

- **Agent (Gemini 2.5 Pro)** receives a natural-language task prompt;
- **Connects** to the server via stdio, calls named tools;
- **Loop** runs until model stops calling tools or hits 15-turn limit;
- **Judge (Gemini 3 Flash)** receives the full trajectory and final answer.

13–25 tasks per API
265 total tasks (~19 avg)



Task Evaluation Example

Task Prompt (sent to agent)

Search eBay for 'wireless headphones'. Return the title, price, and item ID of the first 5 results.

Tools Available (sent to Judge)

- **browse_search_items:** Search listings
- **browse_get_item:** Get details by ID
- ... and other specialized tools

Agent Trajectory & Final Answer (sent to Judge)

```
[Step 1]: browse_search_items({"q": "wireless headphones",  
"limit": 5})
```

```
Result: {"total": 7, "itemSummaries": [{"itemId":  
"v1|110589360302|0",  
  "title": "Wireless Earbuds [...] 30H Playtime",  
  "price": {"value": "29.99", "currency": "USD"}, ...},  
...]}
```

Final Answer: Here are the first 5 results for "wireless headphones" on eBay:

1. Wireless Earbuds – \$29.99 – v1|110589360302|0 [...]

Judge Evaluation

OUTCOME: **PASS**

FAILURE_CAUSE: NONE

SERVER_SUFFICIENT: YES

REASON: The agent correctly identified the search tool, used appropriate parameters, and extracted requested info.

Evaluation based on task completion (PASS/FAIL), root cause analysis of failures, and tool/environment sufficiency.

Data Curation

14 REST APIs across varied domains: productivity, e-commerce, developer tools, communication, payments



 OpenWeatherMap

 ZULIP

ALPHA VANTAGE



ebay

stripe

 Confluence

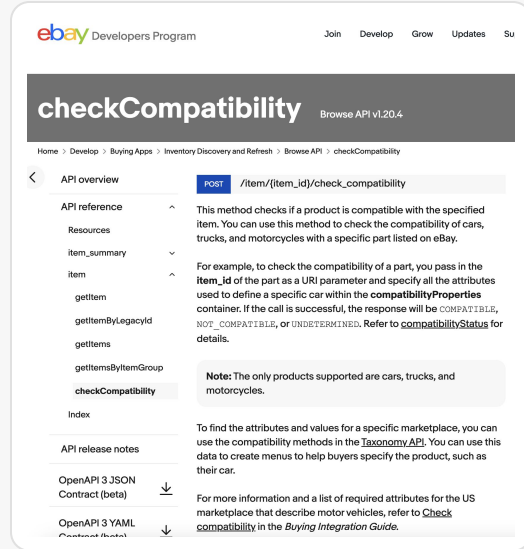


Jira



For each API, we scrape online documentation to obtain messy, (somewhat) unstructured Markdown files.

API docs snapshot example



Depending on the API docs navigation, scrappers may map one endpoint to one .md doc or combine multiple endpoints into one doc.

Excerpt from

~/ebay_buy/docs/api_buy_browse_resources_item_methods_checkCompatibility.md

Sample 1: Checks for Part Compatibility on US Marketplace

Sample 2: Sandbox Sample

Thank you for helping us to improve the eBay developer program.

POST/item/{item_id}/check_compatibility

This method checks if a product is compatible with the specified item. You can use this method to check the compatibility of cars, trucks, and motorcycles with a specific part listed on eBay. For example, to check the compatibility of a part, you pass in the `item_id` of the part as a URI parameter and specify all the attributes used to define a specific car within the `compatibilityProperties` container. If the call is successful, the response will be `COMPATIBLE`, `NOT_COMPATIBLE`, or `UNDETERMINED`. Refer to `compatibilityStatus` for details.

API docs mutations

Mutation mapping

```
{
  "description": "eBay Buy API parameter mutations for
parametric contamination testing.",
  "mutations": [
    {"original": "limit",    "mutated": "max_results"},
    {"original": "offset",   "mutated": "skip"},
    {"original": "sort",     "mutated": "order_by"},
    {"original": "filter",   "mutated": "search_filter"},
    {"original": "item_id",  "mutated": "listing_id"}
  ]
}
```

Evaluation tasks design

Task Structure Example

```
id: search_products
prompt: |
  Search for "wireless headphones" on eBay.
  Report the title, price, and item ID.
expected: itemId # hint for judge
tags: [read]
```

13–25 tasks per API
265 total tasks (~19 avg)

Design Principles (via LLM generation)

- Describe outcomes, never name specific tools or parameters.
- Include API constraints the agent cannot derive (e.g., "amounts are in cents").
- Only include values unreachable via tools (test tokens, sandbox IDs).
- Ask for comparison, counting, or filtering over raw API echoes.
- Capped at 6 sequential calls; optimal range is 2–4.

Results: Key Questions



Can LLMs synthesize functional API tool servers from documentation?



Do synthesized servers enable effective agent task completion?



To what extent do synthesizers rely on parametric knowledge rather than documentation?



How does documentation availability affect synthesis and agent performance?

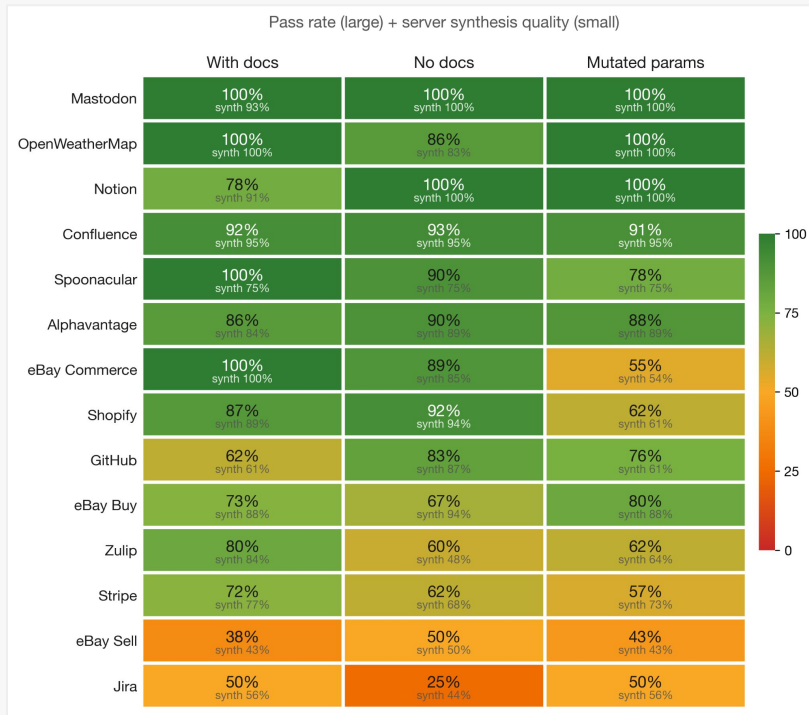


Does synthesis quality predict agent task performance?

Results



Can LLMs synthesize functional API tool servers from documentation?



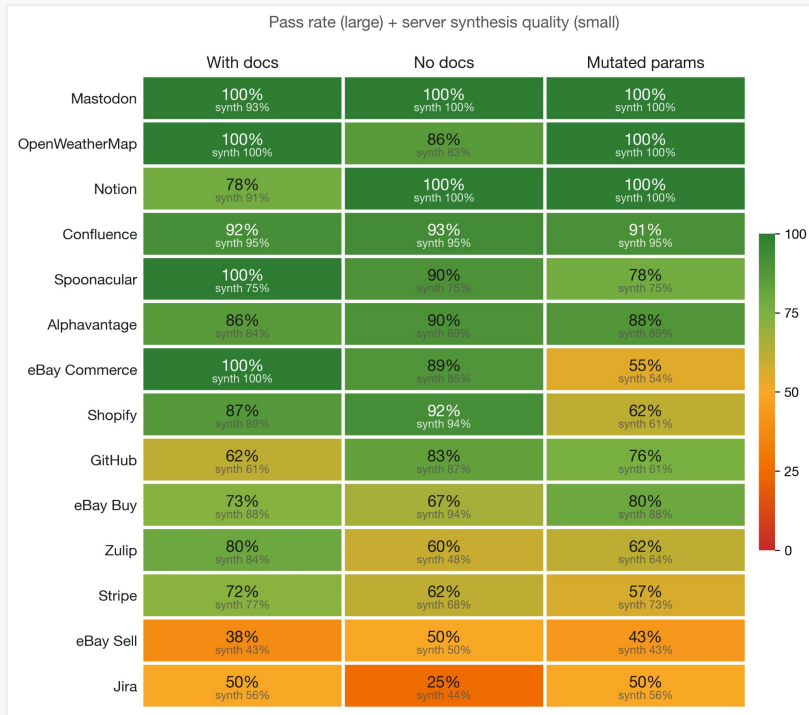
- Synthesis is feasible for most APIs (SYNTH $\geq 70\%$ on 11/14) but degrades on large, less-popular APIs, and failure mode depends on the synthesis model.
- Synthesis remains feasible even without docs for well-known APIs (GitHub nodocs SYNTH 87%)

[@modelcontextprotocol/server-github](#) (official): 65% PASS / 65% SYNTH
13 pass, 7 fail, 3 undefined (M=23 tasks)
[zulipchat-mcp v0.6.2](#) (community): 68% PASS / 68% SYNTH
15 pass, 7 fail, 3 undefined (M=25 tasks)

Results



Do synthesized servers enable effective agent task completion?



- Task completion is generally successful with a median PASS of ~80% across 14 APIs, and synthesized servers match or exceed human-written references on GitHub and Zulip.
- Where PASS is low, SYNTH is also low, indicating synthesis rather than agent capability is the bottleneck.

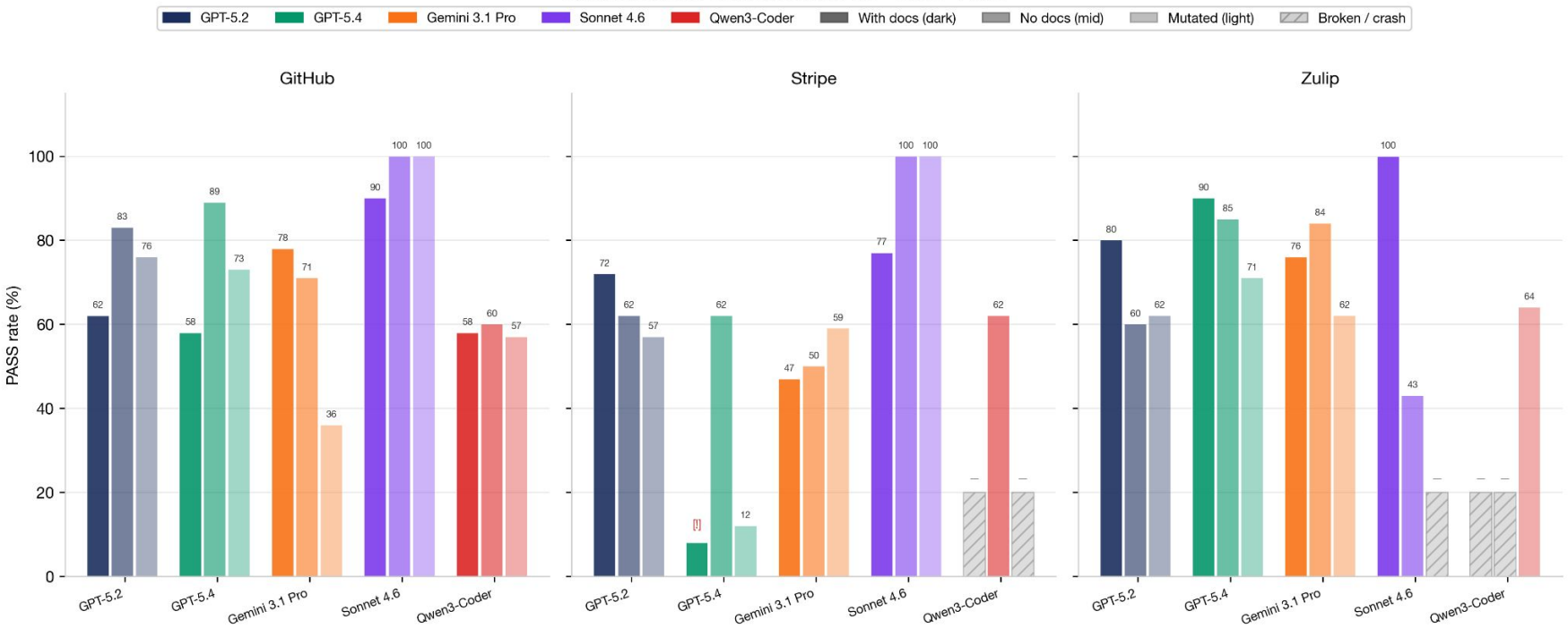
[@modelcontextprotocol/server-github](#) (official): 65% PASS / 65% SYNTH
13 pass, 7 fail, 3 undefined (M=23 tasks)
[zulipchat-mcp v0.6.2](#) (community): 68% PASS / 68% SYNTH
15 pass, 7 fail, 3 undefined (M=25 tasks)

Results



Do synthesized servers enable effective agent task completion?

Synthesis model comparison (GitHub · Stripe · Zulip)

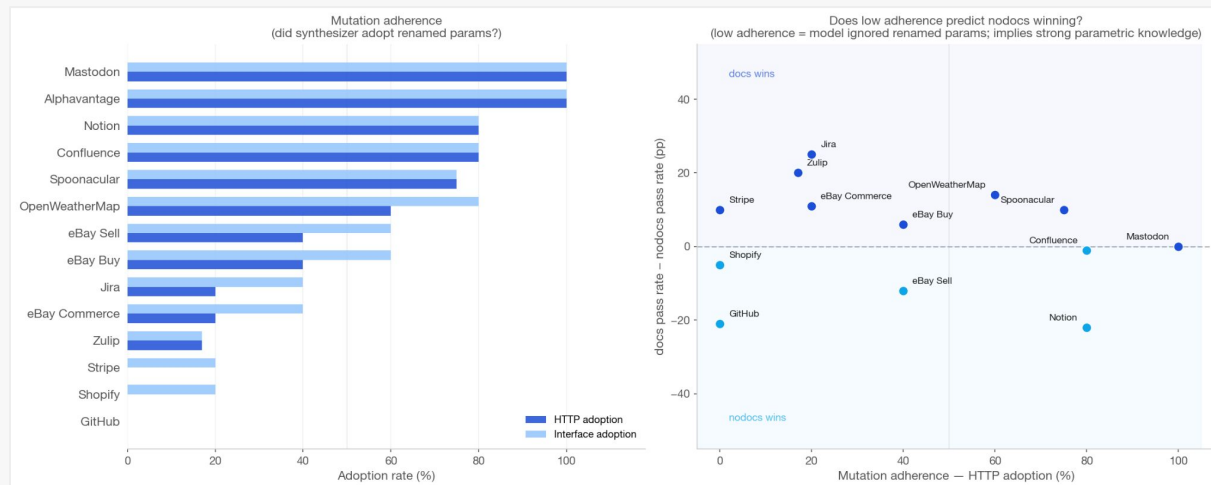


Results



To what extent do synthesizers rely on parametric knowledge rather than documentation?

Nodocs condition is competitive, suggesting models have strong parametric knowledge about the APIs. We deliberately introduce a direct **memory vs. context conflict** via mutated parameter names to check how models resolve it.



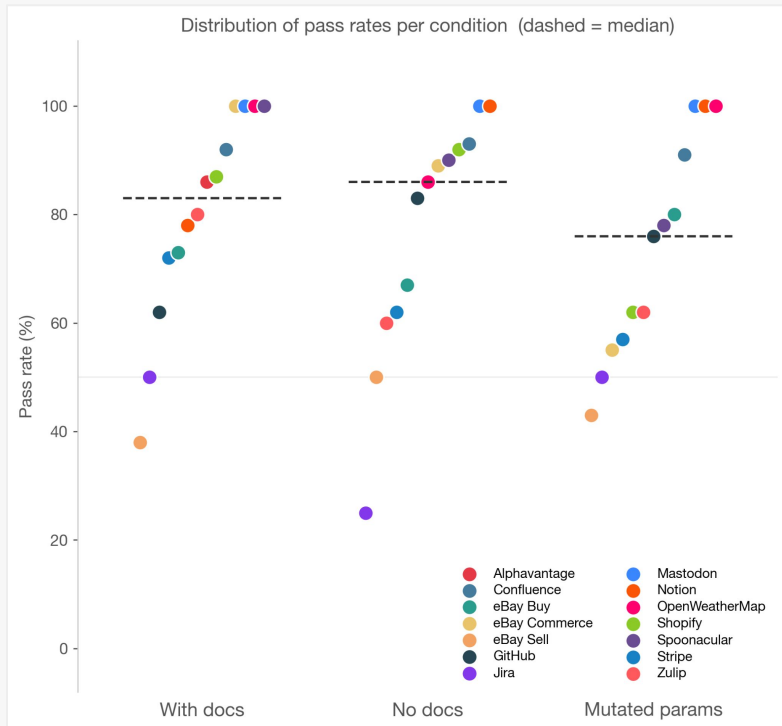
For some APIs, there is zero mutation adherence. **API complexity** is a confound here.

Low adherence does not cleanly predict nodocs winning. GitHub (low adherence, nodocs wins) fits the hypothesis; Notion (high adherence, nodocs wins anyway) does not.

Results



How does documentation availability affect synthesis and agent performance?

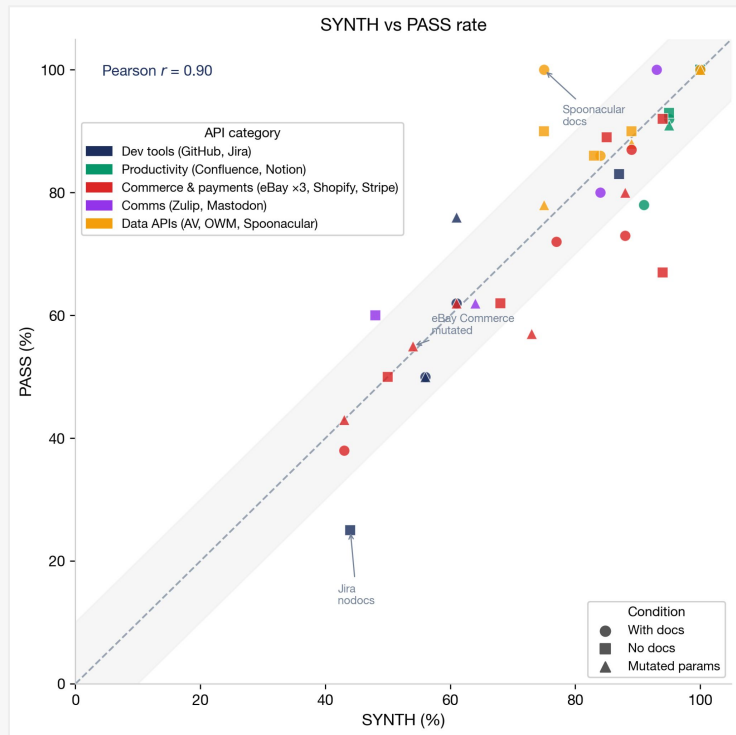


- Stripping documentation increases the median pass rate from 83% (With Docs) to 86% (No Docs).
 - For high-frequency APIs (e.g. GitHub, Notion), large context windows add **processing noise**; internal weights generate cleaner, more functional (or at least on par) tools.
 - Docs provide measurable benefit only for **less familiar APIs** (Zulip) and **more complex APIs** (Stripe).

Results



Does synthesis quality predict agent task performance?



- Strong correlation evidence ($r=0.895$) between SYNTH and PASS.
- When tool coverage is sufficient, agents largely succeed; when it is not, they cannot compensate.

Going Forward

Hybrid Engineering for Context-Memory Conflicts

Tool synthesis operates in two distinct regimes: **Zero-Memory** (proprietary APIs) and **High-Exposure** (public APIs with internal memory competition).



Docs can add unnecessary noise, but bypassing documentation leaves systems fragile. Model parametric gravity generates plausible but outdated code, ignoring new information.



Gated Context Calibration

Live Parametric Verification Gateway

- Run a rapid micro-mutation probe to test memory alignment before full tool synthesis;
- Ration documentation to minimize token noise if memory aligns, but inject full context if a proprietary schema or version drift is detected.



Neuro-Symbolic Enforcement

Hybrid pipeline for strict contracts

- Offload high-level semantic logic to the LLM;
- Route the output through local validation layers (like Pydantic or Context-Free Grammars) to mathematically force compliance with the runtime API contract.

Methodological Contribution

Current Trend

Standard benchmarks evaluate autonomous agents based purely on high-level task outcomes, treating the underlying tool infrastructure as a given.

Synthesis Reality

End-to-end task completion checks cannot distinguish if a model successfully compiled a new interface specification or fell back on training-time knowledge. Synthesized tools can pass a test while containing severe, brittle protocol defects underneath.



The Big-Picture Shift

We shift the focus back to the core challenge of tool creation by establishing an executable benchmark that directly analyzes the synthesized server code.

Intercepting and auditing hidden token alignment provides a reliable metric to check instruction-following in autonomous infrastructures.



Thank you!